Thông tin đề thi

Course: SDN302 **Semester**: FALL 2025 **Tools**: Visual Studio Code, MongoDBCompass, Postman, MongoDB V6 or later

INSTRUCTIONS

Read the instructions carefully before you begin working on the questions.

- Sharing data with others through any device is **NOT allowed**.
- Your development tools must be **Visual Studio Code**, **MongoDBCompass**, **Postman**, and **MongoDB V6 or later**.
- StudentCode: e.g., **se171234**
- Yourdomain: hostname+port; e.g., localhost:5000

IMPORTANT – before working on your solution, you MUST complete the following steps:

1. Before submitting your work, make sure the submitted PE has no syntax error(s).
2. All data must be obtained from the provided JSON files.
3. **If your PE program contains syntax errors or compilation errors, you will not pass the PE requirements, and the mark will be 0.**
4. Create a new project in Visual Studio Code named **SDN302_FA25_StudentCode**.
5. Create a Mongo database named **SDN302_FA25_StudentCodeDB**.
6. **Your submission will be invalid (0 points) if it contains irrelevant content. Store your MongoDB connection string, port value, and JWT secret key in the .env file; failure to do so will result in 0 points.**

Mô tả bài toán

Build a part of the **FOODEX Management System**. The scenario is as follows: A nation has many foods, and each food belongs to one nation.

The database must contain **3 collections** with the following schema:

```
nations ({
    nationName: {type: String, required: true, unique: true},
    continent: { type: String, required: true},
}, { timestamps: true, });

foods ({
    foodName: { type: String, required: true, unique: true},
    foodDescription: { type: String, required: true},
    imageUrl: {type: String, required: true},
    calories: {type: Number, required: true, min: 700, max: 1500},
    rating: {type: Number, required: true, min: 1, max: 5},
    isVegetarian: {type: Boolean, default: false},
    nation: {type: mongoose.Schema.Types.ObjectId, ref: "nation", required: true},
}, { timestamps: true, });

users ({
    name: { type: String, required: true},
    key: { type: String, required: true},
}, { timestamps: true, });
```

Task 01 (1.0 mark)

Use the Express generator to develop the new REST API server, create the related nations, and migrate them into a MongoDB database named **SDN302_FA25_StudentCodeDB**.

Task 02 (5.0 marks)

Create a new REST API server for the **nation** collection, using **JSON Web Token (JWT)** to authenticate the API. The secret key of JWT must be **your StudentCode!@ (e.g., se171234!@) and store in the .env file**. The route path will match requests to:

- Visit **http://yourdomain/auth/signin** to generate the JWT. The account credentials are stored in the users.json file (name: foodreviewer, key: 123456789)
- **http://yourdomain/api/v1/nations** and **http://yourdomain/api/v1/nations/:id** for all methods related to nations.
- Implement CRUD actions with **nation collection**.
- For the DELETE operation, ensure a nation cannot be deleted if any foods are associated with it (i.e., if there are documents in the food collection referencing the nation's _id). Return an appropriate error message (e.g., "Cannot delete nation because it has associated foods") if deletion is attempted in such cases.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

Task 03 (4.0 marks)

Client application for food management

Students must choose one of the template engines (EJS, HANDLEBARS, or PUG) with Express for building the views.

1. **Authentication function**

   At the login view, the route path will match requests to: **http://yourdomain/auth/login**. Users must enter their account name (*name*) and key (*key*) correctly (encrypted with bcrypt and stored in the 'users' collection). Authentication is required for all CRUD actions. Error messages must be displayed when login fails. Account data is stored in the 'users' collection from 'users.json'. After a successful login, users are redirected to the foods view.
2. After successful login, display a list of all foods (using a **card view**, not a table view): foodName, rating, calories, isVegetarian and including the nation name. This route path will match requests to **http://yourdomain/page/foods**.
3. After successful login, allow deletion of a selected food by confirming the action and notifying the result. This route path will match requests to **http://yourdomain/page/foods/:id**.
4. After successful login, add a new food that meets the requirements using a modal form.
   - All fields are required.
   - The *foodName* must contain only letters (a-z, A-Z) and spaces, and it must be unique.
   - The *calories* must be a number between 700 and 1500.
   - The *rating* is entered as a numerical rating, which is then displayed as a corresponding number of stars. E.g: {'rating': 3} => ★★★
   - The *isVegetarian* field must be a toggle or switch control.
   - The *nation* field must be a select control, with options displaying nationName values from the nation collection.
   - This route path will match requests to **http://yourdomain/page/foods**.
5. After successful login, allow updating the food list. All controls must meet the same requirements as the add function. This route path will match requests to **http://yourdomain/page/foods/:id**.

-The end of test-